Component Optimization

SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

https://softuni.bg

Have a Question?

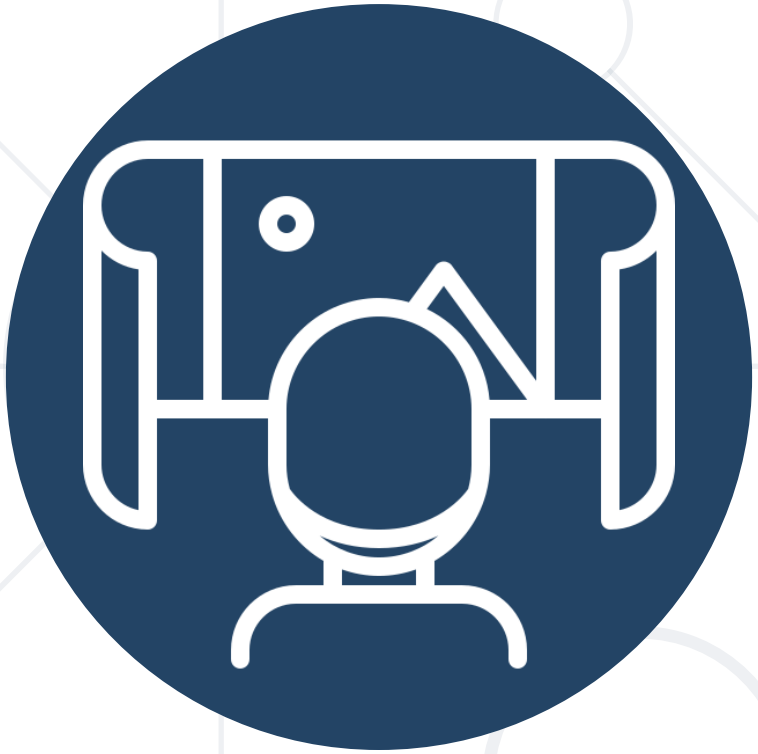
Software University

sli.do

#react

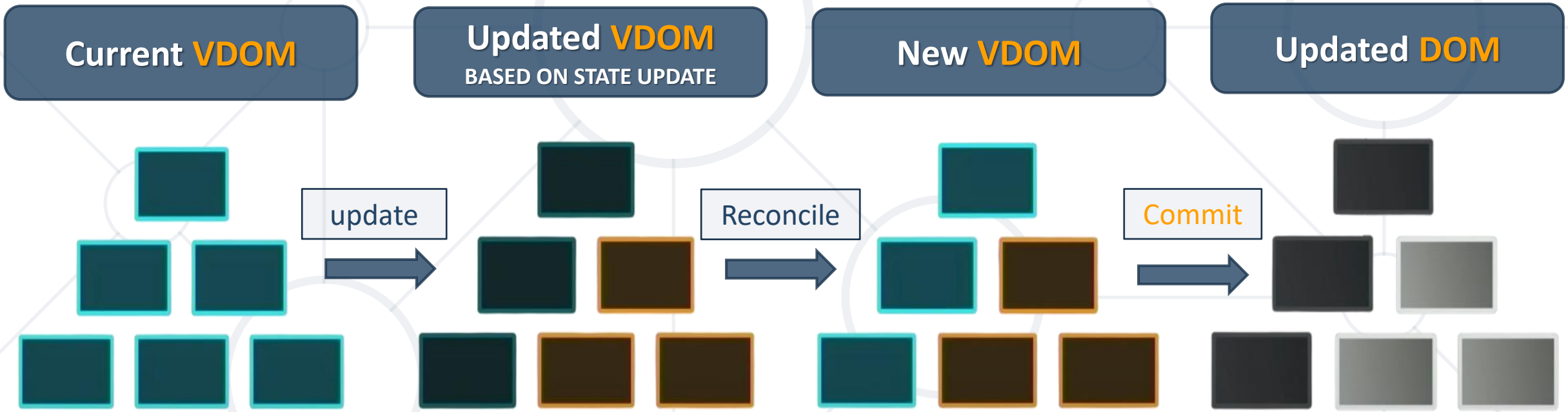
Table of Contents

1. Virtual DOM
2. Profiler
3. React.memo
4. useCallback
5. useMemo
6. Compiler



Virtual DOM

Flow


Current VDOM
Updated VDOM
BASED ON STATE UPDATE
New VDOM
Updated DOM
update
Reconcile
Commit

Profiler

Profiler

- <**Profiler**> lets you measure rendering performance of a React tree programmatically
- Helps identify **performance** bottlenecks and optimize rendering efficiency
  - Wrap a **component tree** with <Profiler> to measure its rendering performance

```
<Profiler id="App" onRender={onRender}>
  <App />
</Profiler>
```

Profiler Props

- A string identifying the part of the UI you are measuring (**id**)
- Callback function executed when components in the profiled tree update (**onRender**)
- Wrap a **component tree** with <Profiler> to measure its rendering performance

```
<Profiler id="App" onRender={onRender}>
  <App />
</Profiler>
```

onRender

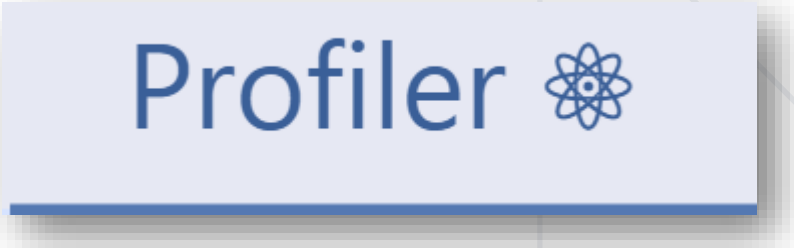
- React calls **onRender** with information about each render cycle
  - **id**: The id prop of <Profiler>
  - **phase**: "mount", "update", or "nested-update"
  - **actualDuration**: Time (ms) spent rendering the profiled tree
  - **baseDuration**: Estimated time to render the entire subtree without optimizations
  - **startTime**: Timestamp when rendering began
  - **commitTime**: Timestamp when React committed the update

React Developer Tools Profiler

- A built-in feature of **React DevTools** for interactive performance profiling
  - Shows component render **durations** and commit times
  - Highlights components that rendered during each **commit**
  - Displays flame **graphs** and ranked **views** for performance insights

React Developer Tools Profiler Use

- Open React Developer Tools in your **browser**
- Navigate to the **Profiler** tab
- Click the **Record** button and interact with your application
- Stop recording to analyze render performance
- Use the flame graph to identify costly re-renders



React.memo

memo

- **memo** lets you skip re-rendering a component when its props are unchanged
- Wrap a component in **memo** to get a **memoized** version of that component
- **returns** a new React component
- It behaves the same as the component provided to memo except that React will not always re-render it when its parent is being re-rendered unless its props have **changed**

memo

- **Memoized** components skip re-renders during parent updates if their new props are the same as the old props

```
const Child = React.memo(function Child({ data }) {
      console.log("Child re-renders");
      // Only logs if `data` changes
return
      <div>{data}</div>;
 });
```

useCallback

useCallback

- **useCallback** lets you cache a a function between re-renders, returning the same function instance unless dependencies change
- **Prevents** unnecessary re-renders in child components (e.g., when passing callbacks to React.memo components)
- Dependency array must **include** all variables used inside the function

useCallback

```
const Parent = () => {
 const [count, setCount] = useState(0);
 // Without useCallback, `handleClick` is recreated on every render:
const handleClick = useCallback(() => {
 setCount(c => c + 1); }
, []); // Same function unless dependencies change
return <Child onClick={handleClick} />;
 };
 const Child = React.memo(({ onClick }) => (
 <button onClick={onClick}>Click</button>
));
```

useMemo

useMemo

- **useMemo** lets you cache the result of a calculation between re-renders, re-calculating it only when dependencies change
- Avoids **expensive** recalculations on every render
- Prevents unnecessary re-renders in child components when passing non-primitive values (e.g., objects, arrays)
- Used for **costly** computations (e.g., filtering/sorting large lists)

useMemo

```
const List = ({ items, filterTerm }) => {
const filteredList = useMemo(() => {
 return
items.filter(item => item.includes(filterTerm));
//Only re-runs if `items` or `filterTerm` changes
}, [items, filterTerm]);

return <Child list={filteredList} />;
 };

const Child = React.memo(({ list }) => ( /* ... */ ));
```

Compiler

React Compiler

- A **compiler** tool that automatically optimizes React apps by
  - **Auto-memoizing** components, values, and functions
  - Eliminating unnecessary re-renders (like manual **useMemo**/**useCallback**)
  - Reducing bundle size via dead code removal
- Still **experimental**

How to Use the React Compiler

- Install packages

```
npm install babel-plugin-react-compiler
```

- Add the compiler plugin to your Babel config (e.g., babel.config.js)

```
module.exports = {
      plugins: [ ['babel-plugin-react-compiler’],
// ...other plugins
      ],
 };
```

- Enable Strict Mode

How to Use the React Compiler

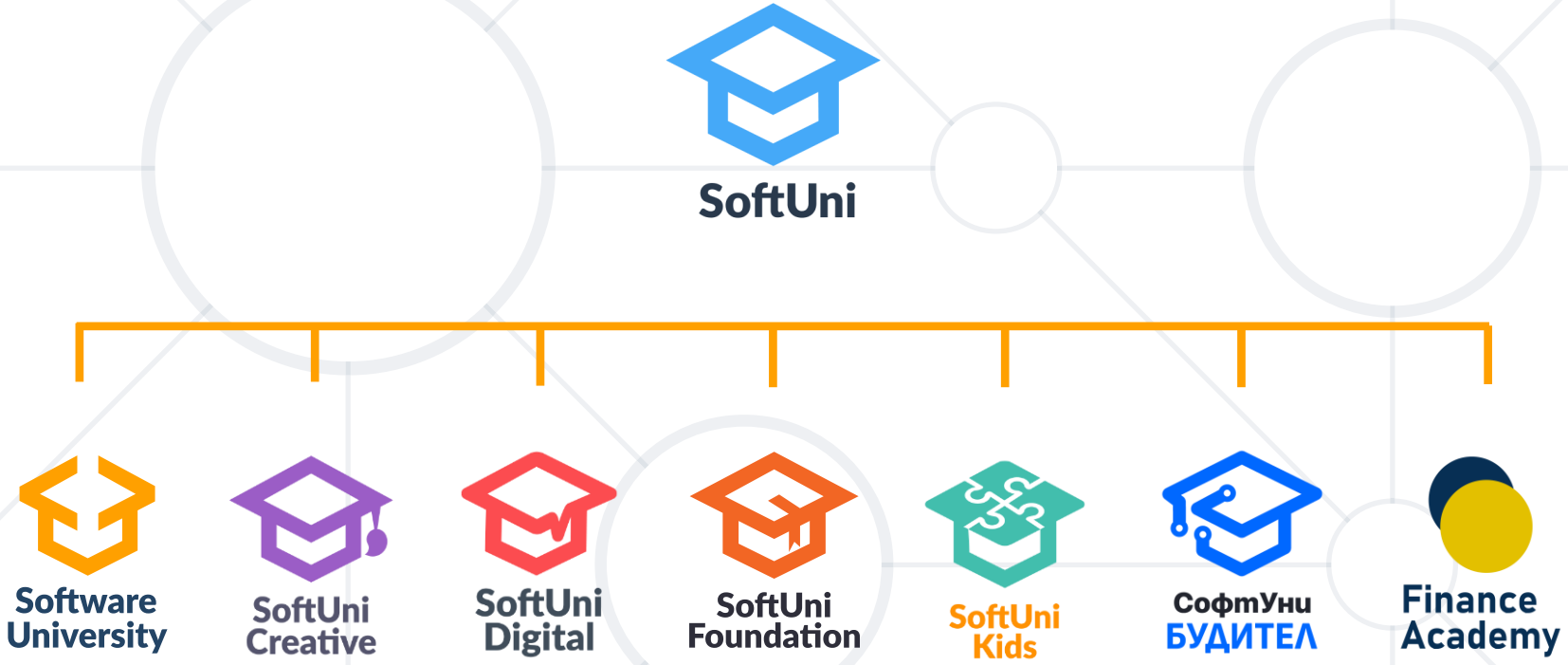
- The compiler automatically optimizes components, values, and functions

```
// No need for manual `useMemo`/`useCallback`!
 const MyComponent = ({ items }) => {
 const filteredList = items.filter(item => item.active);
//Auto-memoized
return
 <Child list={filteredList} />;
};
```

Summary

- **Profiler** find performance bottlenecks
- **React.memo** prevent re-renders of functional components if props are unchanged
- **useCallback** lets you cache a function definition between re-renders
- **useMemo** lets you cache the result of a calculation between re-renders
- **Compiler** replaces manual useMemo/useCallback, but is experimental for now

Questions?



© SoftUni – https://about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

SoftUni Diamond Partners

Software University

SmartIT

Coca-Cola HBC Bulgaria

SUPER HOSTING .BG

createX

Postbank

top EMPLOYER България Bulgaria 2024 FOR A BETTER WORLD OF WORK

DRAFT KINGS

THE CROWN IS YOURS

VIVACOM

INDEAVR

Serving the high achievers

AMBITIONED

PHAR VISION

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg, about.softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg
- © Software University – https://softuni.bg